

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-208БВ-25

Студент: Никонова А.Э.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 02.10.26

Москва, 2026

Постановка задачи

Вариант 16.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в одной из двух операционных систем. В результате работы программа (основной процесс) должен создать для решения задачи один или несколько дочерних процессов. Взаимодействие между процессами осуществляется через системные сигналы/события и/или каналы (pipe). Необходимо обрабатывать системные ошибки, которые могут возникнуть в результате работы.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; — создаёт дочерний процесс.
- `int pipe(int *fd)`; — создаёт однонаправленный канал; `fd[0]` — конец для чтения, `fd[1]` — конец для записи.
- `int dup2(int oldfd, int newfd)`; — перенаправляет `newfd` на тот же открытый объект, что и `oldfd`.
- `int open(const char *path, int flags, mode_t mode)`; — открывает/создаёт файл для записи.
- `ssize_t read(int fd, void *buf, size_t n)`; — читает данные из дескриптора.
- `ssize_t write(int fd, const void *buf, size_t n)`; — пишет данные в дескриптор.
- `int close(int fd)`; — закрывает файловый дескриптор.
- `int execl(const char *path, const char *arg0, ..., NULL)`; — заменяет образ процесса на другую программу.
- `pid_t wait(int *status)`; — ожидает завершения дочернего процесса.
- `void _exit(int status)`; — аварийное завершение процесса без обработчиков `atexit`.
- `ssize_t read(int fd, void *buf, size_t n)`; — читает строки из `stdin` (перенаправлен на `pipe1`).
- `ssize_t write(int fd, const void *buf, size_t n)`; — пишет валидные строки в `stdout` и сообщения об ошибках в `err_fd`.
- `int close(int fd)`; — закрывает write-конец `pipe2`, чтобы родитель получил EOF.
- `void _exit(int status)`; — аварийный выход при ошибке `dup2`.

Я реализовала взаимодействие двух процессов через неименованные каналы. Задача состояла в том, чтобы написать две отдельные программы: родительский процесс `parent` и дочерний процесс `child`, которые обмениваются данными через два канала `pipe`. Родитель спрашивает у пользователя имя файла, открывает этот файл на запись с помощью системного вызова `open` с флагами `O_WRONLY`, `O_CREAT` и `O_TRUNC`, после чего перенаправляет свой стандартный поток вывода в этот файл через `dup2`, так что весь дальнейший вывод родителя и унаследованный вывод ребёнка попадают именно в файл, а не на терминал. Затем родитель создаёт два канала: `pipe1` для передачи строк от родителя к ребёнку и `pipe2` для передачи сообщений об ошибках от ребёнка к родителю. После этого через `fork` создаётся дочерний процесс, в котором закрываются ненужные концы каналов, стандартный ввод перенаправляется на читающий конец `pipe1` через `dup2`, а номер записывающего конца `pipe2` преобразуется в строку и передаётся дочерней программе как аргумент командной строки при вызове `execl`, который заменяет образ текущего процесса на программу `child`. В родителе после `fork` закрываются концы `p1[0]` и `p2[1]`, после чего начинается цикл чтения строк из стандартного ввода функцией `read_line`, которая читает строку посимвольно через `read`,

динамически расширяет буфер через `realloc` и возвращает строку вместе с завершающим символом новой строки, если он был. Каждая прочитанная строка целиком записывается в `p1[1]` с помощью цикла по `write`. Когда пользователь завершает ввод нажатием `Ctrl+D`, `read_line` возвращает `NULL`, родитель закрывает `p1[1]`, и ребёнок получает признак конца файла при чтении из своего стандартного ввода. Дочерняя программа `child`, получив номер дескриптора для вывода ошибок через `argv[1]`, разбирает его вручную функцией `to_int` и в цикле читает строки из своего стандартного ввода, который через `dup2` указывает на читающий конец `pipe1`. Для каждой строки определяется последний значимый символ без учёта символа новой строки: если это точка или точка с запятой, строка признаётся валидной и выводится в стандартный поток вывода ребёнка, который наследуется от родителя и потому указывает на открытый файл; в противном случае в дескриптор ошибок, то есть в записывающий конец `pipe2`, отправляется сообщение с префиксом `INVALID` и самой строкой, а при необходимости добавляется символ новой строки. Для всех записей используется вспомогательная функция `write_all`, которая в цикле вызывает `write` до тех пор, пока не запишет все запрошенные байты или пока `write` не вернёт ошибку. Когда родитель заканчивает ввод и закрывает `p1[1]`, цикл чтения в ребёнке завершается, ребёнок закрывает дескриптор ошибок, и родитель, читающий из `p2[0]`, получает все накопленные сообщения об ошибках и выводит их в свой стандартный поток вывода, то есть в тот же файл; после получения признака конца файла на `p2[0]` родитель закрывает этот дескриптор и вызывает `wait`, чтобы дожидаться завершения дочернего процесса. В итоге в файле оказываются все валидные строки, записанные ребёнком напрямую, и все невалидные строки с префиксом `INVALID`, переданные через `pipe2` и выведенные родителем; порядок между этими двумя группами строк может отличаться от порядка ввода, поскольку валидные строки пишутся ребёнком сразу, а невалидные проходят дополнительный путь через канал и родителя. В работе были использованы системные вызовы `read` и `write` для обмена данными, `open` и `close` для работы с файлом и дескрипторами, `dup2` для перенаправления стандартных потоков, `pipe` для создания каналов, `fork` для создания дочернего процесса, `execl` для запуска отдельной программы, `wait` для синхронизации с ребёнком и `_exit` для аварийного завершения в ветке ребёнка при ошибке. Программа решает поставленную задачу: строки, оканчивающиеся на точку или точку с запятой, считаются валидными и попадают в файл, а все остальные сопровождаются пометкой `INVALID` и также оказываются в файле через родительский процесс.

Код программы

Родитель (parent.c)

```
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <sys/types.h>

#define BUF_SIZE 4096
static char *read_line(int fd)
{
    size_t cap = 128, len = 0;
    char *buf = malloc(cap);
    if (!buf) return NULL;

    char c;
    ssize_t n;
```

```

while ((n = read(fd, &c, 1)) == 1) {
    if (len + 2 > cap) {
        cap *= 2;
        char *tmp = realloc(buf, cap);
        if (!tmp) { free(buf); return NULL; }
        buf = tmp;
    }
    buf[len++] = c;
    if (c == '\n') break;
}
if (n <= 0 && len == 0) { free(buf); return NULL; }

buf[len] = '\0';
return buf;
}

int main(void)
{
    char name[512];
    write(STDOUT_FILENO, "Enter file name: ", 17);

    size_t i = 0;
    char c;
    while (read(STDIN_FILENO, &c, 1) == 1 && c != '\n' && i < sizeof(name) - 1)
        name[i++] = c;
    name[i] = '\0';

    int fd = open(name, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        write(STDERR_FILENO, "Cannot open file\n", 17);
        return 1;
    }
    if (dup2(fd, STDOUT_FILENO) < 0) {
        write(STDERR_FILENO, "dup2 failed\n", 12);
        return 1;
    }
    close(fd);
    int p1[2], p2[2];
    if (pipe(p1) < 0 || pipe(p2) < 0) {
        write(STDERR_FILENO, "pipe failed\n", 12);
        return 1;
    }

    pid_t pid = fork();
    if (pid < 0) {
        write(STDERR_FILENO, "fork failed\n", 12);
        return 1;
    }

    if (pid == 0) {
        close(p1[1]);
        close(p2[0]);
    }

```

```

if (dup2(p1[0], STDIN_FILENO) < 0) _exit(1);
close(p1[0]);

char p2fd_str[16];
int len = 0;
int fdnum = p2[1];
char tmp[16]; int t = 0;
if (fdnum == 0) tmp[t++] = '0';
while (fdnum > 0) { tmp[t++] = '0' + fdnum % 10; fdnum /= 10; }
while (t > 0) p2fd_str[len++] = tmp[--t];
p2fd_str[len] = '\0';

execl("./child", "child", p2fd_str, NULL);
_exit(1);
}

close(p1[0]);
close(p2[1]);

char *line;
while ((line = read_line(STDIN_FILENO)) != NULL) {
    size_t len = strlen(line);
    size_t written = 0;
    while (written < len) {
        ssize_t w = write(p1[1], line + written, len - written);
        if (w <= 0) break;
        written += (size_t)w;
    }
    free(line);
}
close(p1[1]);

char buf[BUF_SIZE];
ssize_t n;
while ((n = read(p2[0], buf, sizeof(buf))) > 0) {
    write(STDOUT_FILENO, buf, (size_t)n);
}
close(p2[0]);

wait(NULL);
return 0;
}

```

Ребенок (child.c)

```

#include <unistd.h>
#include <stdlib.h>
#include <string.h>

#define BUF_SIZE 4096

static char *read_line(int fd)
{

```

```

size_t cap = 128, len = 0;
char *buf = malloc(cap);
if (!buf) return NULL;

char c;
ssize_t n;
while ((n = read(fd, &c, 1)) == 1) {
    if (len + 2 > cap) {
        cap *= 2;
        char *tmp = realloc(buf, cap);
        if (!tmp) { free(buf); return NULL; }
        buf = tmp;
    }
    buf[len++] = c;
    if (c == '\n') break;
}
if (n <= 0 && len == 0) { free(buf); return NULL; }
buf[len] = '\0';
return buf;
}

static int to_int(const char *s)
{
    int v = 0;
    while (*s >= '0' && *s <= '9') v = v * 10 + (*s++ - '0');
    return v;
}

static void write_all(int fd, const char *buf, size_t len)
{
    size_t off = 0;
    while (off < len) {
        ssize_t w = write(fd, buf + off, len - off);
        if (w <= 0) break;
        off += (size_t)w;
    }
}

int main(int argc, char **argv)
{
    if (argc < 2) return 1;
    int err_fd = to_int(argv[1]);

    char *line;
    while ((line = read_line(STDIN_FILENO)) != NULL) {
        size_t len = strlen(line);

        size_t check_len = len;
        if (check_len > 0 && line[check_len - 1] == '\n')
            check_len--;

        int valid = 0;
        if (check_len > 0) {

```

```
    char last = line[check_len - 1];
    if (last == '.' || last == ';') valid = 1;
}

if (valid) {
    write_all(STDOUT_FILENO, line, len);
} else {
    const char *msg = "INVALID: ";
    write_all(err_fd, msg, strlen(msg));
    write_all(err_fd, line, len);
    if (len == 0 || line[len - 1] != '\n')
        write_all(err_fd, "\n", 1);
}
free(line);
}
close(err_fd);
return 0;
}
```

Протокол работы программы

./parent

Enter file name: out.txt

Hello world.

This is bad

Another line;

o

Out.txt: Hello world.

Another line;

INVALID: This is bad

INVALID:

INVALID:

INVALID: o

Вывод

В ходе лабораторной работы я освоила межпроцессное взаимодействие в операционной системе Linux и научилась управлять каналами. Одновременно сложно и интересно было строить взаимодействия родителя и ребенка в соответствии с заданием. Очень долго разбиралась, как передавать информацию между ними и какие функции для этого использовать без некоторых стандартных библиотек.